

Experiment 3

Problem Statement:

Data Collection - Capture social media data from platforms (e.g., Twitter, Facebook, LinkedIn, YouTube) using APIs such as Twitter API v2, Facebook Graph API, and YouTube Data API, along with scraping and crawling techniques (e.g., BeautifulSoup, Selenium)

Theory:

Data cleaning – It is a crucial step in data preprocessing that ensures the dataset is free from inconsistencies, missing values, and irrelevant information. Clean data improves the accuracy and efficiency of analytical models, making the results more reliable. This experiment focuses on data cleaning techniques using pandas, NumPy, and other relevant libraries, followed by storing the cleaned dataset in a structured database.

How to Perform Data Cleanliness –

The process begins by thoroughly understanding data and its structure to identify issues like missing values, duplicates and outliers. Performing data cleaning involves a systematic process to identify and remove errors in a dataset. The following are essential steps to perform data cleaning.



- **Removal of Unwanted Observations:** Identify and remove irrelevant or redundant (unwanted) observations from the dataset. This step involves analyzing data entries for duplicate records, irrelevant information or data points that do not contribute to analysis and prediction. Removing them from the dataset helps reduce noise and improves the overall quality of the dataset.

- **Fixing Structure errors:** Address structural issues in the dataset such as inconsistencies in data formats or variable types. Standardized formats ensure uniformity in data structure and hence data consistency.
- **Managing outliers:** Outliers are those points that deviate significantly from dataset mean. Identifying and managing outliers significantly improves model accuracy as these extreme values influence analysis. Depending on the context, decide whether to remove outliers or transform them to minimize their impact on analysis.
- **Handling Missing Data:** To handle missing data effectively we need to impute missing values based on statistical methods, removing records with missing values or employing advanced imputation techniques. Handling missing data helps prevent biases and maintain the integrity of data.

Throughout the process documentation of changes is crucial for transparency and future reference. Iterative validation is done to test effectiveness of the data cleaning resulting in a refined dataset and can be used for meaningful analysis and insights.

Tools Used –

- pandas: A Python library for data manipulation and analysis.
- NumPy: A library for numerical operations and handling missing values.
- Regular Expressions (re): Used for text cleaning by removing unwanted symbols.
- NLTK & spaCy: Libraries for natural language processing (NLP) tasks, including tokenization and lemmatization.
- MongoDB/PostgreSQL: Databases for storing structured and unstructured data.
- Cloud Storage (Google Cloud Storage, AWS S3): For remote data storage and access.

Steps Explained –

1. Data Cleaning Steps

a) Loading the Dataset

The dataset is first loaded using pandas:

```
import pandas as pd
df_comments = pd.read_csv("drive/MyDrive/dataset/merged_comments.csv")
```

b) Converting Text to Lowercase

Converting all text data to lowercase ensures uniformity and reduces redundancy.

```
df_comments['Comment'] = df_comments['Comment'].str.lower()
```

c) Removing Unwanted Symbols

Special characters and punctuation marks are removed using regular expressions:

```
import re
df_comments['Comment'] = df_comments['Comment'].apply(lambda x: re.sub(r'[^\w]', '', x))
```

d) Tokenization

Breaking text into individual words:

```
df_comments['Comment'] = df_comments['Comment'].apply(lambda x: x.split())
```

e) Lemmatization

Reducing words to their base form to avoid redundancy:

```
from nltk.stem import WordNetLemmatizer
nltk.download('wordnet')
lemmatizer = WordNetLemmatizer()
df_comments['Comment'] = df_comments['Comment'].apply(lambda x:
[lemmatizer.lemmatize(word) for word in x])
```

f) Removing Stop Words

Common words that do not contribute to meaning are removed:

```
from nltk.corpus import stopwords
nltk.download('stopwords')
stop_words = set(stopwords.words('english'))
df_comments['Comment'] = df_comments['Comment'].apply(lambda x: [word for word in x if
word not in stop_words])
```

g) Storing Cleaned Data

After cleaning, the dataset is converted back to a readable format and saved:

```
df_comments['Comment'] = df_comments['Comment'].apply(lambda x: ' '.join(x))
df_comments.to_csv('drive/MyDrive/dataset/preprocessed_comments.csv', index=False)
```

Storage Explained –

a) Relational Database (MySQL)

A structured approach is required for storing the cleaned dataset:

- Define a schema with appropriate data types.
- Load data into the database using **SQLAlchemy** or **MySQL Connector**.

```
CREATE TABLE comments (
    id INT AUTO_INCREMENT PRIMARY KEY,
    comment_text TEXT
);
import mysql.connector
conn = mysql.connector.connect(user='user', password='password', host='localhost',
database='database')
cursor = conn.cursor()
for _, row in df_comments.iterrows():
    cursor.execute("INSERT INTO comments (comment_text) VALUES (%s)",
(row['Comment'],))
conn.commit()
cursor.close()
conn.close()
```

Data cleaning is an essential step in data preprocessing that enhances data quality for analysis. This experiment demonstrated various techniques such as text normalization, tokenization, lemmatization, and stopword removal. Additionally, different storage solutions were explored, ensuring accessibility and structured organization of data.